JAVASCRIPT BASICS

OBJECTS, ARRAYS, AND FUNCTIONS

PREPARED BY:
ADRIAN BENEDICT C. ENOC



LEARNING OBJECTIVES

01 Define JavaScript ES (ECMAScript) and explain its purpose in JavaScript development.

02 Define and explain variables and objects in JavaScript and describe their roles in storing and organizing data.

03 Define JavaScript arrays and explain how they are used to store and manage multiple values in a single variable.

04 Define JavaScript functions and explain how they are used to perform reusable tasks and organize code.

WHAT IS JAVASCRIPT

A programming language and core technology of the web platform, alongside HTML and CSS.

Key Features of JavaScript

- **Lightweight and Interpreted** – Runs directly in browsers without requiring compilation first.
- **Dynamic Typing** – Variables can hold different types of values without explicitly declaring their data type.
- **Object-Oriented** – Supports objects, properties, methods, and object-based programming.
- **Event-Driven** – Responds to user actions such as clicks, keyboard input, and mouse movements.
- **Cross-Platform** – Runs on different operating systems and environments, including browsers and servers.
- **First-Class Functions** – Functions can be stored in variables, passed as arguments, and returned from other functions.
- **Asynchronous Programming** – Supports asynchronous operations using callbacks, Promises, and async/await.
- **DOM Manipulation** – Allows developers to dynamically modify HTML elements and webpage content.
- **Large Ecosystem** – Has many libraries and frameworks such as React, Angular, and Vue.js.
- **Server-Side Development** – Can be used outside the browser with environments such as Node.js.

WHAT IS JAVASCRIPT ES6

ECMAScript 6, also known as ES6 or ECMAScript 2015, a significant update to the JavaScript Language that introduced many new features, making JavaScript more powerful and efficient.

Key Features of ES6

1. **let and const** – Provides better ways to declare variables with block scope.
2. **Arrow Functions** (=>) – Provides a shorter syntax for writing functions.
3. **Template Literals** – Makes it easier to create strings and embed variables using ${}.
4. **Default Parameters** – Allows functions to have default values for parameters.
5. **Destructuring** – Allows values to be easily extracted from arrays and objects.
6. **Spread and Rest Operators (...)** – Used to expand or collect values in arrays, objects, and function parameters.
7. **Classes** – Provides a cleaner syntax for creating objects and implementing object-oriented programming.
8. **Modules (import / export)** – Allows code to be separated into reusable files and components.
9. **Promises** – Provides a structured way to handle asynchronous operations.
10. **Enhanced Object Literals** – Provides shorter and more convenient syntax for creating objects.

VARIABLES

Variables in JavaScript are used to store data values.

Declaration syntax: (old way)

```
// old declaration
var a = 10;
var  name = “Enoc”
```

Declaration syntax: (Newer way) ES6 and later

```
// let - Block-scoped variable
let count = 5;
count = 6;
console.log(count); // output 6
```

What is ES6, and why is it continually being mentioned in this topic?

ES6 is important because it significantly modernized JavaScript and established features that are still widely used in current JavaScript development.

VARIABLES

Variables in JavaScript are used to store data values.

Declaration syntax: (old way)

```
// old declaration
var a = 10;
var  name = “Enoc”
```

Declaration syntax: (Newer way) ES6 and later

```
// let - Block-scoped variable
let count = 5;
count = 6;
console.log(count); // output 6
```

What is ES6, and why is it continually being mentioned in this topic?

ES6 is important because it significantly modernized JavaScript and established features that are still widely used in current JavaScript development.

VARIABLES (CONT)

JavaScript Variables can be declared in 4 ways:

Variable	Example
automatically	x=1
var	var number = 1;
let	let isLoggedIn = true;
const	const language = "en"

OBJECTS

Objects hold data in key-value pairs. They are useful for representing structured data.

Declaration syntax:

```
let person = { name: "Adrian", age: 23, isStudent: true };
console.log(person.name); // Access property → Adrian
person.age = 24; // Modify property
let user = { firstName: "Adrian",
             greet: function()
             { return "Hello, " + this.username; }
           };
console.log(user.greet()); // Hello, Adrian
```

ARRAYS

Arrays store multiple values in an ordered list.

Declaration syntax:

```
let fruits = ["apple", "banana", "mango"];
console.log(fruits[0]);     // apple
fruits.push("orange");      // Add element
 fruits.pop();                // Remove last element
console.log(fruits.length); // Count items
fruits.forEach(function(fruit) {
console.log(fruit); }); // prints all of the fruits

for (let fruit of fruits) {
    console.log(fruit);
}
```

FUNCTIONS

Functions lets you reuse blocks of code

There are 3 ways to declare a functions

1. Function Declaration
2. Function Expression
3. Arrow functions

Type	Syntax	ES6?
Function Declaration	function greet() {}	No
Function Expression	const greet = function() {}	No
Arrow Function	const greet = () => {}	**Yes**

FUNCTION DECLARATION

A function is declared using the function keyword.

Declaration Syntax:

```
1   function greet() {
2       console.log("Hello!");
3   }
4
5   greet();
```

FUNCTION EXPRESSION

A function is assigned to a variable.

Declaration Syntax:

```
const greet = function() {
    console.log("Hello BSIT")
}
```

ARROW FUNCTION

Arrow Functions allow a shorter syntax for function expressions. An arrow function uses the => symbol.

Declaration Syntax:

```
1   const greet = () => {
2       console.log("Hello!");
3   };
```

Thank You!